ICS 344 - 01

AYMAN AL JOHANI - 202016260

DVSA URL: http://dvsa-website-928747700347-us-east-1.s3-website.us-east-1.amazonaws.com/

AWS Region: us-east-1

02/05/2026

Lesson #7: Over-Privileged Functions

Lesson #7: Over-Privileged Functions

Part 1) Goal and Vulnerability Summary

This lesson shows a problem called over-privileged functions in a serverless AWS setup. In this case, the Lambda function is given more permissions than it actually needs, such as AWSLambda_FullAccess and AmazonDynamoDBFullAccess. Because of that, the function can do many actions that are not part of its main job. For example, it could read, change, or even delete data, which becomes dangerous if the function is misused or attacked. The main issue here is that the permissions are too wide and not limited properly, which goes against the principle of least privilege.

Part 2) Why This Works / Root Cause

The main reason this problem happens is that the Lambda function is given too many permissions through its IAM role. Instead of giving it only what it needs, it has wide permissions like AWSLambda_FullAccess and AmazonDynamoDBFullAccess. Because of that, the function can do actions that are not really required and could be risky. If someone manages to exploit the function, they can use these extra permissions to access or change resources that should not be accessible.

Part 3) Environment and Setup

The demo was done using a serverless application on AWS. The main part we looked at was a Lambda function that uses an IAM role with very broad permissions like AWSLambda_FullAccess and AmazonDynamoDBFullAccess. This function also works with DynamoDB as the backend database. To analyze the issue, I checked the IAM role and its permissions using the AWS Console. The test was done using a normal user account (team-member), without any special privileges. I mainly used the AWS Console to review roles and policies, and CloudWatch logs to observe how the Lambda function behaves.

Part 4) Reproduction Steps

1. Open the AWS Management Console and go to IAM.

2. From the left menu, click on "Roles" and look for the Lambda execution role related to the DVSA application (for example, roles that include "Admin" or similar names).

3. Open the role and go to the "Permissions" tab to see the attached policies.

4. Check the policies and notice that it includes permissions like AWSLambda_FullAccess and AmazonDynamoDBFullAccess.

5. Click on one of these policies (for example, AmazonDynamoDBFullAccess) and review what actions are allowed. You will see that it allows all actions (dynamodb:*).

6. This shows that the role has very broad access, more than what the Lambda function actually needs.

7. (You can also open the IAM Policy Simulator, select the same role, and test actions like Scan or DeleteTable in DynamoDB. You will see that these actions are allowed.

These steps confirm that the Lambda function has excessive permissions, which means it is over-privileged.

Part 5) Evidence and Proof

I confirmed this vulnerability by checking the IAM role attached to the Lambda function using the AWS Console. I noticed that the role includes very broad permissions like AWSLambda_FullAccess and AmazonDynamoDBFullAccess. When I looked into these policies, it was clear that they allow almost everything. For example, AmazonDynamoDBFullAccess allows all DynamoDB actions (dynamodb:*), which means the function can read, write, and even delete data from any table. Also, AWSLambda_FullAccess gives full control over Lambda functions. Since these policies are directly attached to the role, it means the Lambda function is actually running with these high permissions. If the function is compromised, these permissions can be misused.

Evidence includes:

- Screenshot of the IAM role showing attached policies (AWSLambda_FullAccess and AmazonDynamoDBFullAccess).

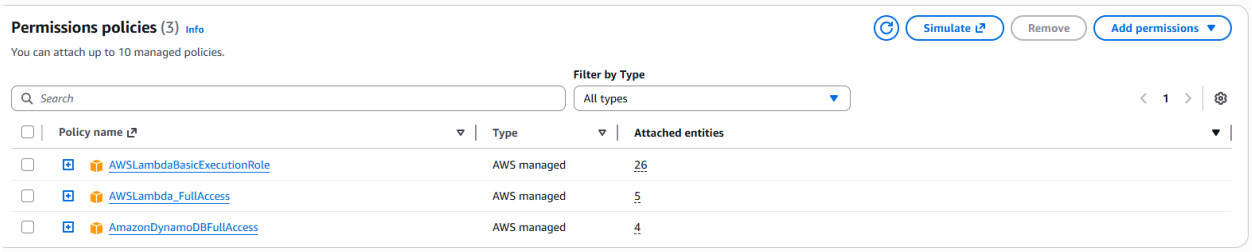

Figure 1: IAM role showing attached FullAccess managed policies (AWSLambda_FullAccess and AmazonDynamoDBFullAccess).

- Screenshot of the IAM policy details showing unrestricted actions (e.g., dynamodb:*).

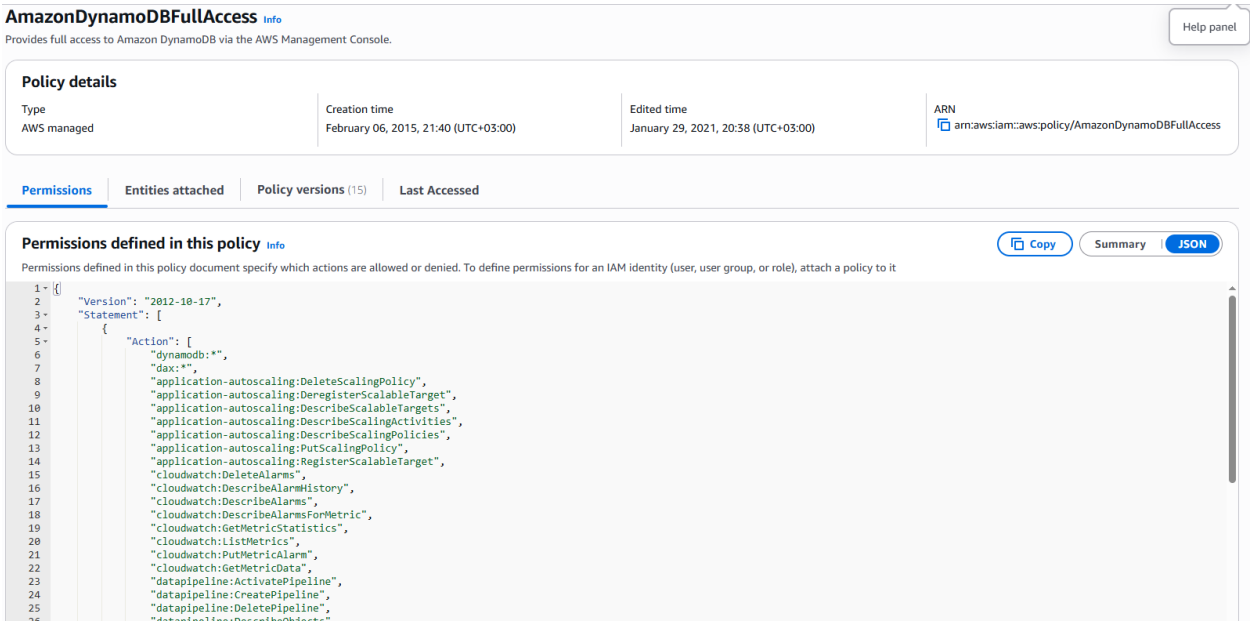

Figure 2: IAM policy details showing unrestricted DynamoDB actions (dynamodb:*), allowing full access to all database operations.

- An additional proof of the vulnerability is observed in the IAM role trust relationship, which shows that the role can be assumed by the Lambda service ("lambda.amazonaws.com"). This confirms that any Lambda function using this role will inherit its excessive permissions, making the vulnerability exploitable in practice.

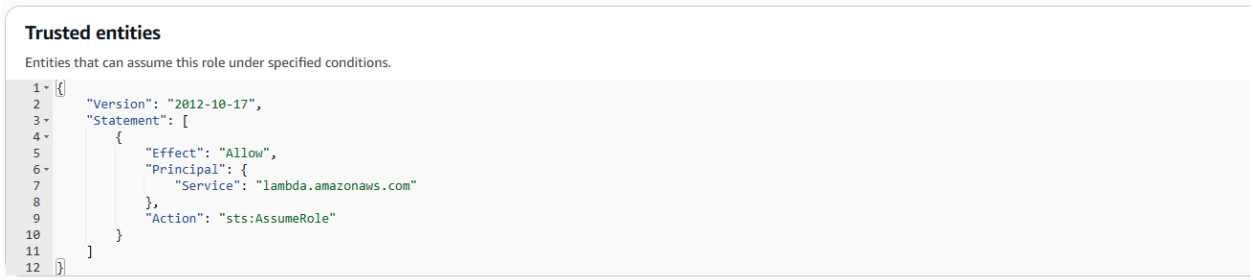

Figure 3: IAM role trust relationship showing that the role can be assumed by the Lambda service (lambda.amazonaws.com).

The IAM Policy Simulator confirms that the Lambda execution role allows high-risk actions such as dynamodb:Scan and DeleteItem. This demonstrates that the role has excessive permissions beyond its intended functionality.

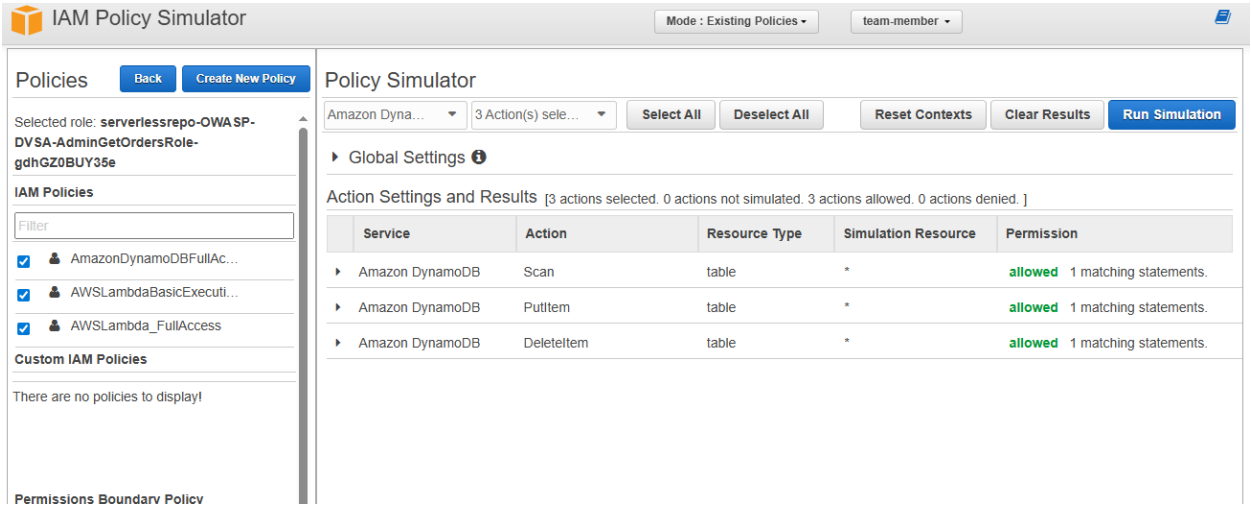

Figure 4: IAM Policy Simulator showing that the Lambda execution role allows DynamoDB actions such as Scan, PutItem, and DeleteItem (result: Allowed).

Part 6) Fix Strategy / Probable Mitigation

To fix this issue, I applied the principle of least privilege to the IAM role used by the Lambda function. Instead of keeping the broad permissions like AWSLambda_FullAccess and AmazonDynamoDBFullAccess, I removed them. Then, I created a custom policy that only allows the minimum required actions, such as DynamoDB read operations like GetItem or Query on a specific table. This way, the Lambda function can only do what it actually needs and nothing more. As a result, even if the function is exploited, the damage will be limited because it no longer has access to unnecessary or sensitive resources.

Part 7) Code / Config Changes

File / resource changed: IAM role attached to the Lambda function (for example: serverlessrepo-OWASP-DVSA-AdminGetOrdersRole)

Before (overly permissive configuration):

The Lambda role had very broad permissions, including:

- AWSLambda_FullAccess

- AmazonDynamoDBFullAccess

These policies allowed the function to do almost anything on Lambda and DynamoDB resources. In other words, it had full access ("Action": "*", "Resource": "*").

After (restricted least-privilege configuration):

I removed these overly permissive policies and replaced them with a custom inline policy that only allows the necessary actions.

Example of the new policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "dynamodb:GetItem",
        "dynamodb:Query"
      ],
      "Resource": "arn:aws:dynamodb:region:account-id:table/OrdersTable"
    }
  ]
}
```

With this change, the Lambda function can only read data from a specific table instead of having full access to everything. This removes unnecessary and risky permissions, like deleting tables or modifying data. This also reduces the attack surface and makes the system safer.

Part 8) Verification After Fix

After applying the least privilege policy, I repeated the same checks on the IAM role. This time, the role no longer had broad permissions like AWSLambda_FullAccess or AmazonDynamoDBFullAccess. Instead, it only allows limited DynamoDB actions such as GetItem and Query on a specific table.

Because of that, risky actions like deleting tables or scanning all data are no longer allowed. This shows that the extra permissions were successfully removed and the issue is fixed.

At the same time, the Lambda function still works normally, since it keeps the basic permissions it needs to read data.

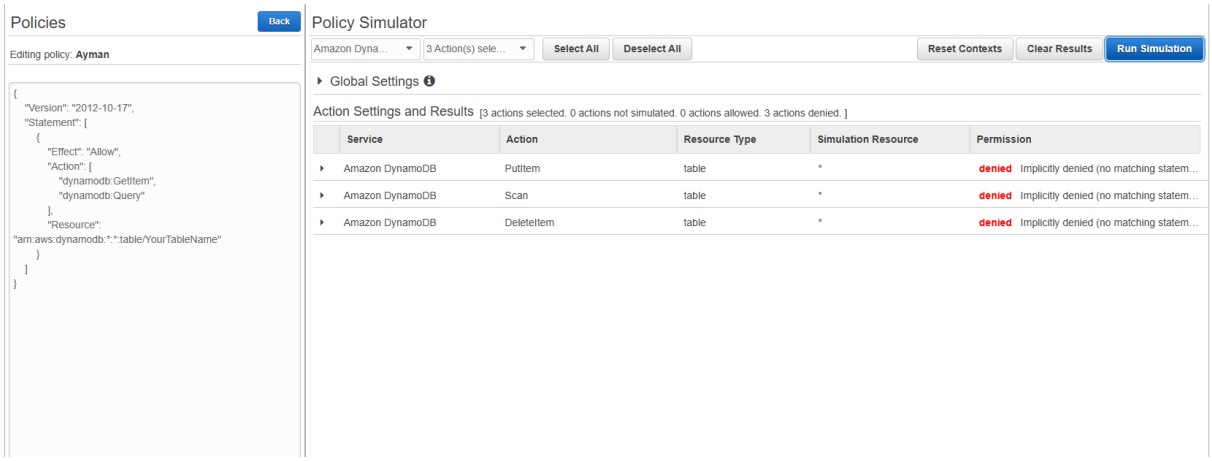

Figure 5: IAM Policy Simulator results showing that high-risk actions are denied after applying least privilege.

Part 9) Structured Operation and Security Analysis

Table A — Intended logic, artifacts, and behavior comparison

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Over-Privileged Functions	The Lambda function should follow the principle of least privilege and only have the permissions it actually needs (for example, read-only access to a specific DynamoDB table).	IAM role configuration, attached policies (AWSLambda_FullAccess, AmazonDynamoDBFullAccess), policy JSON showing (dynamodb:*), and AWS Console screenshots.	In a normal case, the IAM role should only allow limited actions such as GetItem or Query on a specific table.	In this case, the IAM role includes FullAccess policies, which allow unrestricted actions (dynamodb:*) on all resources, going beyond what the function actually needs.

Table B — Deviation analysis, fix, and verification

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Over-Privileged Functions	The permissions given to the Lambda function are more than what it actually needs, which breaks the principle of least privilege.	Misconfiguration	I fixed this by updating the IAM role, removing the FullAccess policies, and replacing them with a limited policy that only allows necessary actions like	After the fix, only basic read actions are allowed, and risky actions like DeleteTable are no longer permitted. This shows that the issue	Not applicable

			DynamoDB GetItem and Query.GetItem, Query only).	has been resolved.	

9.1 Intended Logic and Security Rule(s)

Under normal conditions, a user sends requests to a backend Lambda function, which then processes the request and reads data from DynamoDB. The function is supposed to do only the specific tasks it needs, like reading data from a certain table.

The IAM role attached to the Lambda function should limit what the function can do, and only allow the minimum required actions.

Security rules:

- The system should only give the Lambda function the permissions it actually needs.

- It should not allow wildcard permissions like "*" for actions or resources.

- The Lambda function should only access specific resources related to its job.

- All access to AWS services should follow the principle of least privilege.

In the correct setup, the Lambda function can do its job (like reading data), but it cannot perform unnecessary or risky actions

.9.2 Evidence Sources and Behavior Trace

I based my analysis on several sources, such as the IAM role configuration, attached policies, policy JSON details (like dynamodb:*), and screenshots from the AWS Console. These helped me understand both how the system is supposed to work and how it actually behaves.

Normal behavior:

In a secure setup, the IAM role should only allow limited actions like GetItem or Query on a specific DynamoDB table. This ensures that the Lambda function only performs what it is supposed to do.

Exploit behavior:

In this case, the IAM role has very broad permissions like AWSLambda_FullAccess and AmazonDynamoDBFullAccess. The policy JSON shows (dynamodb:*), which means full access to all database actions. This goes against the intended security rules and allows actions that should not be allowed.

Post-fix behavior:

After applying the least privilege policy, the IAM role is limited to specific actions such as GetItem and Query on a single table. Risky actions like DeleteTable or full scans are no longer allowed, which shows that the issue has been fixed.

9.3 Deviation Analysis and Classification

The exploit behavior is different from what the system is supposed to do. The Lambda function should only have limited permissions, like read-only access to a specific DynamoDB table. However, in this case, the IAM role has very broad permissions such as AWSLambda_FullAccess and AmazonDynamoDBFullAccess, which allow all actions (dynamodb:*).

This is clearly shown in the IAM policies and screenshots, where full access and wildcard permissions are enabled. From a security point of view, this is a problem because it increases the attack surface and makes it easier to misuse the system, such as accessing or deleting data without proper control.

Classification: Accidental misconfiguration

9.4 Explainable Fix and Post-Fix Validation

This issue happened because it was assumed that giving the Lambda function broad permissions would make development easier, without thinking about the security risks. However, this led to giving the function more access than it actually needs.

To fix this, I updated the IAM role by removing the overly permissive policies like AWSLambda_FullAccess and AmazonDynamoDBFullAccess. Then, I replaced them with a limited policy that only allows the necessary actions, such as dynamodb:GetItem and dynamodb:Query on a specific table.

After applying the fix, I verified that risky actions like deleting tables or accessing all data are no longer allowed. At the same time, the Lambda function still works correctly and can perform its normal read operations.

Part 10) Takeaway / Lessons Learned

This issue happened because it was assumed that giving broad permissions would not cause serious security problems. In a serverless environment, this is risky because Lambda functions can be a strong entry point to access different cloud resources.

If the function is compromised, it could be used to access data, modify it, or even delete it. That’s why giving extra permissions can be dangerous.

The main lesson here is to always apply the principle of least privilege. Each component should only have the permissions it really needs. It’s also important to review permissions regularly and make sure everything is configured securely to avoid similar issues in the future.